UNIVERSIDAD PERUANA UNIÓN

Desarrollo de Aplicaciones Móviles
Actividad Autónoma N.º 11

Pruebas unitarias y evidencia de escenarios offline

Estudiante: Jorge Armando Quispe Huaman

Unidad 2 - sesión 5 (sesión 11 del curso)

Fecha de ejecución y documento: 23 de octubre de 2026, America/Lima

Proyecto: PharmaMobil sobre PharmaSoft. Plataforma ejecutada: Android.

Rama: feature/s11-autonoma-quispe

https://github.com/123456jor/pharmaMobile-/tree/feature/s11-autonoma-quispe

Versión de entrega: rama publicada en GitHub con autorización del estudiante. Enlace comprobado: HTTP 200 y commit publicado visible.

Alcance acordado: Android y Postman. No se ejecutó iOS ni Swagger. Se declaran esas diferencias frente a la guía en la lista de cotejo.

Resumen verificable

Seis pruebas obligatorias aprobadas en commonTest. Suite completa: 94 pruebas, cero fallos, cero errores y cero omitidas. Compilación del APK debug exitosa. No se necesita red ni emulador para ejecutar los seis casos con MockEngine y SQLite en memoria.

Cinco escenarios ejecutados en Pixel 8 Pro. El producto AA_Autonoma_Jorge_23Oct pasó de ID local -1 y pendiente=1 a ID 83 y pendiente=0; Postman confirmó el recurso con HTTP 200. La base mantiene diez filas, seis activas y cero pendientes al terminar.

Antigüedad: marca persistida tras sincronización completa, texto natural, caso nunca sincronizado y migración real 1 a 2 que conservó los nueve productos anteriores. La evidencia de un día utiliza una edición de depuración declarada de 26 horas; no transcurrió ese tiempo real.

Expiración: JWT genuino del backend con duración temporal de un minuto. Se comprobó exp < ahora antes de abrir. La consulta local conserva datos, borra credenciales vencidas y bloquea cambios; al volver la conexión se pide login. La duración normal de 3600 segundos fue restaurada.

Límites y acciones personales

Postman reemplaza Swagger por instrucción del usuario, por lo que el punto literal de Swagger se marca No. No hay ejecución iOS; el tramo multiplataforma del guion queda parcial. El ensayo en voz alta, comprensión del código, revisión de conclusiones, entrega al aula y sustentación corresponden al estudiante. La publicación fue autorizada y el enlace de GitHub fue verificado.

Entorno

Windows 11; Microsoft JDK 21.0.12.1; Gradle 9.1.0; Kotlin 2.4.10; AGP 9.0.1; SQLDelight 2.1.0; Ktor 3.6.0; coroutines 1.10.2. Emulador API 37.1. Android llama a 10.0.2.2:8080; Postman a localhost:8080. Oracle corre en el contenedor pharmasoft-oracle.

Detalles reproducibles: docs/s11/entorno.md, mapa-codigo.md, matriz.md, consultar_base.py y consultas/. Las horas de bitácora son Lima; el emulador está en GMT y muestra la hora en formato de 12 horas.

Producto 1. Seis pruebas comunes

2. Registro sin red

El motor lanza IOException. Se exige ID negativo, fila pendiente, emisión del Flow y coincidencia con la consulta de pendientes.

4. La bajada conserva pendientes

Se aísla reemplazarSincronizados porque un POST fallido detiene la sincronización completa antes del GET. La lista remota omite el pendiente: deben existir ambas filas y conservarse intacta la local.

1. Sincronización con red

Dos productos remotos, base nueva: el Flow y SQLite deben contener la lista completa, sin pendientes.

3. Envío al reconectar

La misma base crea sin red y sincroniza con red. Se cuenta exactamente un POST aceptado; el ID confirmado es positivo y desaparece la marca pendiente.

5. Identificadores sin colisión

Se comprueban cantidad, nombres y datos de un producto existente del servidor y uno creado localmente tras recibir su ID definitivo.

6. Segunda sincronización

Se comparan listas completas tras dos sincronizaciones y se exige un único POST en total. El segundo ciclo no duplica el registro.

Orden real de escritura: primero 2 y 4 en f49b9cd; después 1, 3, 5 y 6 en d33a5b1. Cada prueba llama conBaseNueva, crea su driver y cierra recursos. La lógica está en commonTest y el actual JDBC en androidHostTest; iOS no fue objeto de esta ejecución.

Sensibilidad y correcciones

Se cambió temporalmente guardarPendiente a pendiente=false. Las pruebas 2 y 4 fallaron con aserciones; se restauró el archivo automáticamente en finally y se verificó que no quedara diferencia. Nunca se confirmó ese defecto. La suite correcta volvió a pasar con 94 casos.

La primera prueba nueva de fecha falló por una respuesta simulada que escribía tamano en lugar del campo real tamanio. Se corrigió el JSON del test sin debilitar aserciones; se conservan salida inicial y salida corregida. No fue necesario alterar el contrato de producción.

Commits de la rama publicada

```
f49b9cd Jorge Armando Quispe Huaman <joorge0148@gmail.com> test: acredita registro sin red y conservacion de pendientes
en actividad autonoma
d33a5b1 Jorge Armando Quispe Huaman <joorge0148@gmail.com> test: verifica sincronizacion completa ids y ausencia de
duplicados
fc3067e Jorge Armando Quispe Huaman <joorge0148@gmail.com> feat: persiste antiguedad de sincronizacion y migra la base
sin perder productos
de29d57 Jorge Armando Quispe Huaman <joorge0148@gmail.com> fix: permite consultar cache sin red con sesion vencida y
bloquea modificaciones
c052b69 Jorge Armando Quispe Huaman <joorge0148@gmail.com> test: acredita sensibilidad del patron offline y 91 pruebas
completas
f9fdf1d Jorge Armando Quispe Huaman <joorge0148@gmail.com> fix: bloquea escritura al caducar y retira sesion previa ante
rechazo
f09721e Jorge Armando Quispe Huaman <joorge0148@gmail.com> docs: entrega bitacora real capturas Postman y PDF de
actividad autonoma 11
```

fc3067e agrega antigüedad y migración; de29d57 corrige consulta con sesión vencida y primera apertura sin red; c052b69 incorpora comprobaciones adicionales, reporte y reevaluación de sesión cuando vence con la app abierta. Los commits de implementación y documentación están disponibles en el enlace de la rama. f9fdf1d refuerza el bloqueo inmediato al vencer la sesión y elimina la sesión anterior ante un 401.

Resultado de sensibilidad (fragmento, salida completa en repositorio)

```
ProductoRepositorioOfflineAutonomaTest > prueba04LaBajadaNoBorraElProductoPendiente FAILED
    java.lang.AssertionError at ProductoRepositorioOfflineAutonomaTest.kt:58
ProductoRepositorioOfflineAutonomaTest > prueba02RegistroSinRedConservaPendienteYLoEmiteElFlow FAILED
    java.lang.AssertionError at ProductoRepositorioOfflineAutonomaTest.kt:46
2 tests completed, 2 failed
> Task :shared:testAndroidHostTest FAILED
BUILD FAILED in 22s
```

Código de las seis pruebas

Código real de ProductoRepositorioOfflineAutonomaTest.kt. La explicación para estudiar cada caso está al final de este documento y en docs/s11/guia-estudio.md.

```
 1 package pe.edu.upeu.pharmamobil.data.repository
 2
 3 import io.ktor.client.engine.mock.MockEngine
 4 import io.ktor.client.engine.mock.respond
 5 import io.ktor.http.HttpHeaders
 6 import io.ktor.http.HttpMethod
 7 import io.ktor.http.headersOf
 8 import kotlinx.serialization.encodeToString
 9 import kotlinx.serialization.json.Json
10 import kotlinx.coroutines.flow.first
11 import kotlinx.coroutines.test.runTest
12 import kotlinx.io.IOException
13 import pe.edu.upeu.pharmamobil.data.local.ProductoLocal
14 import pe.edu.upeu.pharmamobil.data.local.baseEnMemoria
15 import pe.edu.upeu.pharmamobil.data.local.crearDriverEnMemoria
16 import pe.edu.upeu.pharmamobil.data.remote.ProductoApi
17 import pe.edu.upeu.pharmamobil.data.remote.crearHttpClient
18 import pe.edu.upeu.pharmamobil.data.remote.dto.PaginaResponseDto
19 import pe.edu.upeu.pharmamobil.data.remote.dto.ProductoResponseDto
20 import pe.edu.upeu.pharmamobil.domain.model.Producto
21 import kotlin.test.Test
22 import kotlin.test.assertEquals
23 import kotlin.test.assertTrue
24
25 /** Cada caso abre y cierra su propia base; MockEngine evita red y emulador. */
26 class ProductoRepositorioOfflineAutonomaTest {
27     private val nuevo = Producto(0, "Producto local autónoma", 5.5, 15, categoriaId = 1)
28     private val cabeceras = headersOf(HttpHeaders.ContentType, "application/json")
29     private fun Producto.dto() = ProductoResponseDto(id, nombre, precio, stock, estado, categoriaId, categoriaNombre)
30     private fun pagina(productos: List<Producto>) = Json.encodeToString(
31         PaginaResponseDto(productos.map { it.dto() }, 0, 100, productos.size.toLong(), 1, true)
32     )
33
34     private suspend fun conBaseNueva(
35         motor: MockEngine,
36         comprobar: suspend (ProductoRepositorioOffline, ProductoLocal) -> Unit
37     ) {
38         val driver = crearDriverEnMemoria()
39         val local = ProductoLocal(baseEnMemoria(driver))
40         val cliente = crearHttpClient(baseUrl = "http://prueba/api/v1/", engine = motor, esDepuracion = false)
41         try { comprobar(ProductoRepositorioOffline(local, ProductoApi(cliente)), local) }
42         finally { cliente.close(); driver.close() }
43     }
44
45     @Test
46     fun prueba02RegistroSinRedConservaPendienteYLoEmiteElFlow() = runTest {
47         conBaseNueva(MockEngine { throw IOException("Sin conexión") }) { repo, local ->
48             val guardado = repo.registrar(nuevo)
49             val emitido = repo.observar().first().single()
50             assertTrue(guardado.id < 0)
51             assertTrue(emitido.pendiente)
52             assertEquals(guardado, emitido)
53             assertEquals(listOf(guardado), local.pendientes())
54         }
55     }
56
57     @Test
58     fun prueba04LaBajadaNoBorraElProductoPendiente() = runTest {
59         conBaseNueva(MockEngine { throw IOException("Sin conexión") }) { repo, local ->
60             val pendiente = repo.registrar(nuevo)
61             val remoto = nuevo.copy(id = 71, nombre = "Producto del servidor", pendiente = false)
62             // Aislamos la bajada: un POST fallido detiene sincronizar antes del GET.
63             // Probar el reemplazo local acredita que solo elimina pendiente=0.
64             local.reemplazarSincronizados(listOf(remoto))
65             val lista = repo.observar().first()
66             assertEquals(2, lista.size)
67             assertEquals(setOf(pendiente.id, remoto.id), lista.map { it.id }.toSet())
68             assertEquals(pendiente, lista.single { it.id == pendiente.id })
69             assertEquals(listOf(pendiente), local.pendientes())
70         }
71     }
72
73     @Test
74     fun prueba01SincronizarConRedGuardaYEmiteLaListaCompleta() = runTest {
75         val servidor = listOf(nuevo.copy(id = 41, nombre = "Servidor uno"), nuevo.copy(id = 42, nombre = "Servidor
dos"))
```

```
        conBaseNueva(MockEngine { solicitud ->
            assertEquals(HttpMethod.Get, solicitud.method)
            respond(pagina(servidor), headers = cabeceras)
        }) { repo, local ->
            repo.sincronizar()
            val emitidos = repo.observar().first()
            assertEquals(servidor.toSet(), emitidos.toSet())
            assertEquals(2, emitidos.size)
            assertEquals(emitidos, local.listar())
            assertTrue(local.pendientes().isEmpty())
        }
    }

    @Test
    fun prueba03ReconectarEnviaElPendienteYLoConfirmaConIdReal() = runTest {
        var conRed = false
        var postAceptados = 0
        val confirmado = nuevo.copy(id = 72)
        conBaseNueva(MockEngine { solicitud ->
            if (!conRed) throw IOException("Sin conexión")
            if (solicitud.method == HttpMethod.Post) {
                postAceptados++
                respond(Json.encodeToString(confirmado.dto()), headers = cabeceras)
            } else respond(pagina(listOf(confirmado)), headers = cabeceras)
        }) { repo, local ->
            val pendiente = repo.registrar(nuevo)
            assertTrue(pendiente.id < 0 && local.pendientes().single().pendiente)
            // Se conserva la misma base y cambia solo la disponibilidad del servidor.
            conRed = true
            repo.sincronizar()
            assertEquals(1, postAceptados)
            assertTrue(local.pendientes().isEmpty())
            assertEquals(confirmado, repo.observar().first().single())
        }
    }

    @Test
    fun prueba05ProductoLocalYRemotoNoSeSobrescriben() = runTest {
        val existente = nuevo.copy(id = 41, nombre = "Producto del servidor", precio = 8.0)
        val confirmado = nuevo.copy(id = 72)
        conBaseNueva(MockEngine { solicitud ->
            respond(if (solicitud.method == HttpMethod.Post) Json.encodeToString(confirmado.dto())
                else pagina(listOf(existente, confirmado)), headers = cabeceras)
        }) { repo, local ->
            val pendiente = local.guardarPendiente(nuevo)
            local.guardarSincronizado(existente)
            assertEquals(setOf(pendiente.id, 41L), local.listar().map { it.id }.toSet())
            repo.sincronizar()
            val lista = repo.observar().first()
            assertEquals(2, lista.size)
            assertEquals(existente, lista.single { it.id == 41L })
            assertEquals(confirmado, lista.single { it.id == 72L })
        }
    }

    @Test
    fun prueba06SincronizarDosVecesConservaCantidadYNoRepitePost() = runTest {
        var postAceptados = 0
        val confirmado = nuevo.copy(id = 72)
        conBaseNueva(MockEngine { solicitud ->
            if (solicitud.method == HttpMethod.Post) {
                postAceptados++
                respond(Json.encodeToString(confirmado.dto()), headers = cabeceras)
            } else respond(pagina(listOf(confirmado)), headers = cabeceras)
        }) { repo, local ->
            local.guardarPendiente(nuevo)
            repo.sincronizar()
            val primera = repo.observar().first()
            repo.sincronizar()
            val segunda = repo.observar().first()
            assertEquals(1, primera.size)
            assertEquals(primera.size, segunda.size)
            assertEquals(primera, segunda)
            assertEquals(1, postAceptados)
            assertTrue(local.pendientes().isEmpty())
        }
    }
}
```

Salida completa de las pruebas

Transcripción íntegra de docs/s11/salida-pruebas.txt. Comando: gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug --console=plain. La advertencia de iOS corresponde a Windows; no se afirma una ejecución iOS. Los renglones largos se ajustan al papel sin omitir contenido.

```
Reusing configuration cache.
> Task :shared:generateAndroidHostTestAssets UP-TO-DATE
> Task :shared:copyNonXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:generateExpectResourceCollectorsForCommonMain UP-TO-DATE
> Task :shared:androidPreBuild UP-TO-DATE
> Task :shared:preAndroidMainBuild UP-TO-DATE
> Task :shared:convertXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:generateComposeResClass UP-TO-DATE
> Task :shared:generateAndroidMainAssets UP-TO-DATE
> Task :shared:generateCommonMainPharmaDbInterface UP-TO-DATE
> Task :shared:kmpPartiallyResolvedDependenciesChecker
> Task :shared:androidJar UP-TO-DATE
> Task :shared:convertXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :shared:processAndroidHostTestJavaRes NO-SOURCE
> Task :shared:convertXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForCommonMain UP-TO-DATE
> Task :shared:generateAndroidMainEmptyResourceFiles UP-TO-DATE
> Task :shared:writeAndroidMainAarMetadata UP-TO-DATE
> Task :shared:prepareComposeResourcesTaskForAndroidMain NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :androidApp:preBuild UP-TO-DATE
> Task :shared:preAndroidHostTestBuild UP-TO-DATE
> Task :androidApp:preDebugBuild UP-TO-DATE
> Task :shared:copyNonXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:convertXmlValueResourcesForCommonMain NO-SOURCE
> Task :shared:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :androidApp:mergeDebugNativeDebugMetadata NO-SOURCE
> Task :shared:processAndroidMainManifest UP-TO-DATE
> Task :androidApp:generateDebugBuildConfig UP-TO-DATE
> Task :androidApp:generateDebugAssets UP-TO-DATE
> Task :androidApp:javaPreCompileDebug UP-TO-DATE
> Task :shared:generateResourceAccessorsForAndroidMain NO-SOURCE
> Task :shared:generateAndroidHostTestResources UP-TO-DATE
> Task :androidApp:generateDebugResources UP-TO-DATE
> Task :androidApp:desugarDebugFileDependencies UP-TO-DATE
> Task :shared:prepareComposeResourcesTaskForCommonTest NO-SOURCE
> Task :androidApp:createDebugCompatibleScreenManifests UP-TO-DATE
> Task :androidApp:extractDeepLinksDebug UP-TO-DATE
> Task :androidApp:packageDebugResources UP-TO-DATE
> Task :shared:generateResourceAccessorsForCommonTest NO-SOURCE
> Task :shared:mergeAndroidMainJniLibFolders UP-TO-DATE
> Task :androidApp:mergeDebugJniLibFolders UP-TO-DATE
> Task :androidApp:writeDebugAppMetadata UP-TO-DATE
> Task :androidApp:validateSigningDebug UP-TO-DATE
> Task :androidApp:writeDebugSigningConfigVersions UP-TO-DATE
> Task :shared:mergeAndroidMainNativeLibs NO-SOURCE
> Task :shared:checkAndroidHostTestAarMetadata UP-TO-DATE
> Task :shared:copyAndroidMainJniLibsProjectOnly UP-TO-DATE
> Task :shared:prepareComposeResourcesTaskForAndroidHostTest NO-SOURCE
> Task :shared:generateResourceAccessorsForAndroidHostTest NO-SOURCE
> Task :androidApp:checkDebugAarMetadata UP-TO-DATE
> Task :androidApp:processDebugNavigationResources UP-TO-DATE
> Task :androidApp:checkDebugDuplicateClasses UP-TO-DATE
> Task :shared:prepareComposeResourcesTaskForCommonMain UP-TO-DATE
> Task :shared:processAndroidHostTestManifest UP-TO-DATE
> Task :androidApp:mergeDebugNativeLibs UP-TO-DATE
> Task :shared:generateResourceAccessorsForCommonMain UP-TO-DATE
> Task :androidApp:parseDebugLocalResources UP-TO-DATE
> Task :shared:copyAndroidMainComposeResourcesToAndroidAssets UP-TO-DATE
> Task :androidApp:stripDebugDebugSymbols UP-TO-DATE
> Task :androidApp:processDebugMainManifest UP-TO-DATE
> Task :androidApp:compileDebugNavigationResources UP-TO-DATE
> Task :shared:generateActualResourceCollectorsForAndroidMain UP-TO-DATE
> Task :androidApp:processDebugManifest UP-TO-DATE
> Task :androidApp:generateDebugRFile UP-TO-DATE
> Task :shared:mergeAndroidMainAssets UP-TO-DATE
> Task :androidApp:processDebugManifestForPackage UP-TO-DATE
> Task :shared:packageAndroidMainResources UP-TO-DATE
> Task :androidApp:mergeDebugAssets UP-TO-DATE
> Task :shared:compileAndroidMainLibraryResources UP-TO-DATE
> Task :shared:mergeAndroidHostTestAssets UP-TO-DATE
> Task :shared:parseAndroidMainLocalResources UP-TO-DATE
> Task :androidApp:compressDebugAssets UP-TO-DATE
> Task :shared:generateAndroidMainRFile UP-TO-DATE
> Task :shared:mergeAndroidHostTestResources UP-TO-DATE
```

```
> Task :androidApp:mapDebugSourceSetPaths UP-TO-DATE
> Task :shared:mapAndroidHostTestSourceSetPaths UP-TO-DATE
> Task :androidApp:mergeDebugResources UP-TO-DATE
> Task :androidApp:mergeExtDexDebug UP-TO-DATE
> Task :androidApp:processDebugResources UP-TO-DATE
> Task :shared:processAndroidHostTestResources UP-TO-DATE
> Task :shared:packageAndroidHostTestForUnitTest UP-TO-DATE
> Task :shared:generateAndroidHostTestConfig UP-TO-DATE

> Task :shared:compileAndroidMain
w: file:///H:/ProyectosUPEU/pharmaMobile-/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.android.
kt:7:38 'constructor(p0: String!, p1: String!): Locale' is deprecated. Deprecated in Java.
w: file:///H:/ProyectosUPEU/pharmaMobile-/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/App.kt:93:13 'fun
KoinContext(koin: Koin = ..., content: ComposableFunction0<Unit>): Unit' is deprecated. KoinContext is not needed
anymore. This can be removed. Compose Koin context is setup with StartKoin().
w: file:///H:/ProyectosUPEU/pharmaMobile-/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/App.kt:221:44 'val
Icons.Filled.Logout: ImageVector' is deprecated. Use the AutoMirrored version at Icons.AutoMirrored.Filled.Logout.
w: file:///H:/ProyectosUPEU/pharmaMobile-/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/App.kt:238:52 'val
Icons.Filled.Logout: ImageVector' is deprecated. Use the AutoMirrored version at Icons.AutoMirrored.Filled.Logout.

> Task :shared:processAndroidMainJavaRes UP-TO-DATE
> Task :shared:bundleLibRuntimeToDirAndroidMain
> Task :shared:bundleAndroidMainClassesToCompileJar
> Task :shared:bundleAndroidMainClassesToRuntimeJar
> Task :androidApp:compileDebugKotlin UP-TO-DATE
> Task :androidApp:processDebugJavaRes UP-TO-DATE
> Task :androidApp:mergeDebugJavaResource UP-TO-DATE
> Task :androidApp:compileDebugJavaWithJavac UP-TO-DATE
> Task :androidApp:dexBuilderDebug UP-TO-DATE
> Task :androidApp:mergeProjectDexDebug UP-TO-DATE
> Task :androidApp:mergeLibDexDebug
> Task :androidApp:packageDebug
> Task :androidApp:assembleDebug
> Task :androidApp:createDebugApkListingFileRedirect UP-TO-DATE
> Task :shared:compileAndroidHostTest

> Task :shared:testAndroidHostTest

SharedCommonTest > example PASSED

SharedLogicAndroidHostTest > example PASSED

EjecutarLlamadaTest > un401ProtegidoSeConvierteEnSesionExpirada STANDARD_ERROR
    SLF4J(W): No SLF4J providers were found.
    SLF4J(W): Defaulting to no-operation (NOP) logger implementation
    SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

EjecutarLlamadaTest > un401ProtegidoSeConvierteEnSesionExpirada PASSED

EjecutarLlamadaTest > traduce409AlMensajeDelServidor STANDARD_OUT
    REQUEST: http://test/api/v1/productos/99
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    RESPONSE: 409 Conflict
    METHOD: GET
    FROM: http://test/api/v1/productos/99
    COMMON HEADERS
    -> Content-Type: application/json

EjecutarLlamadaTest > traduce409AlMensajeDelServidor PASSED

EjecutarLlamadaTest > traduce404ANoEncontrado STANDARD_OUT
    REQUEST: http://test/api/v1/productos/99
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    RESPONSE: 404 Not Found
    METHOD: GET
    FROM: http://test/api/v1/productos/99
    COMMON HEADERS
    -> Content-Type: application/json

EjecutarLlamadaTest > traduce404ANoEncontrado PASSED

EjecutarLlamadaTest > clienteAdjuntaTokenEnRutaProtegidaPeroNoEnLogin PASSED

EjecutarLlamadaTest > traduce400ConLosErroresPorCampo STANDARD_OUT
    REQUEST: http://test/api/v1/productos/99
    METHOD: GET
```

```
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    RESPONSE: 400 Bad Request
    METHOD: GET
    FROM: http://test/api/v1/productos/99
    COMMON HEADERS
    -> Content-Type: application/json

EjecutarLlamadaTest > traduce400ConLosErroresPorCampo PASSED

AuthRepositorioRestTest > alReconectarSinTokenPideLoginYNoRestauraSesion PASSED

AuthRepositorioRestTest > respuesta401NoUsaLaSesionCacheada PASSED

AuthRepositorioRestTest > login401SeExponeComoCredencialesInvalidasSinGuardarToken PASSED

AuthRepositorioRestTest > primeraAperturaSinTokenYSinRedPermiteConsultaVaciaSinAutenticar PASSED

AuthRepositorioRestTest > guardaTokenTrasLoginYLoUsaParaReconstruirSesion PASSED

AuthRepositorioRestTest > sesionVencidaNoSeRestauraYPermiteSoloConsultaSinToken PASSED

AuthRepositorioRestTest > jwtVencidoNoSeAdjuntaNiDuranteLaComprobacionDeReconexion PASSED

AuthRepositorioRestTest > reabrirSinRedRestauraSoloSesionGuardadaVigente PASSED

ClienteRepositorioEnMemoriaTest > asignaIdCorrelativoAlRegistrar PASSED

ClienteRepositorioEnMemoriaTest > listarDevuelveLosClientesRegistrados PASSED

ProductoRepositorioEnMemoriaTest > listarDevuelveLosProductosRegistrados PASSED

ProductoRepositorioEnMemoriaTest > asignaIdCorrelativoAlRegistrar PASSED

ProductoRepositorioOfflineAutonomaTest > prueba04LaBajadaNoBorraElProductoPendiente PASSED

ProductoRepositorioOfflineAutonomaTest > prueba01SincronizarConRedGuardaYEmiteLaListaCompleta PASSED

ProductoRepositorioOfflineAutonomaTest > prueba05ProductoLocalYRemotoNoSeSobrescriben PASSED

ProductoRepositorioOfflineAutonomaTest > prueba03ReconectarEnviaElPendienteYLoConfirmaConIdReal PASSED

ProductoRepositorioOfflineAutonomaTest > prueba02RegistroSinRedConservaPendienteYLoEmiteElFlow PASSED

ProductoRepositorioOfflineAutonomaTest > prueba06SincronizarDosVecesConservaCantidadYNoRepitePost PASSED

ProductoRepositorioOfflineTest > errorServidorNoSeSilenciaComoSinConexion PASSED

ProductoRepositorioOfflineTest > descargaTodasLasPaginas PASSED

ProductoRepositorioOfflineTest > falloPostConservaPendienteYNoEjecutaGet PASSED

ProductoRepositorioOfflineTest > sincronizarNoDejaDuplicadosNiPendientesHuerfanos PASSED

ProductoRepositorioOfflineTest > subirPendientesOcurreAntesDeBajar PASSED

ProductoRepositorioOfflineTest > idsLocalesSonNegativosDistintosYConservanCategoria PASSED

ProductoRepositorioOfflineTest > sin red el producto queda pendiente y la pantalla lo ve PASSED

ProductoRepositorioOfflineTest > reemplazarSincronizadosPreservaPendientes PASSED

SincronizacionFechaTest > fechaSeGuardaSoloTrasExitoYUnFalloNoLaRenueva PASSED

AppModuleTest > resuelveLosViewModelConSusCasosDeUso PASSED

AppModuleTest > losRepositoriosSonUnicosEnTodaLaAplicacion PASSED

AppModuleTest > resuelveLosRepositoriosPorSuInterfazDeDominio PASSED

AppModuleTest > registraLaImplementacionOfflineDeProductos PASSED

AppModuleTest > resuelveLosCuatroCasosDeUsoConSusRepositorios PASSED

AntiguedadTest > nuncaSincronizado PASSED

AntiguedadTest > segundosYRelojAtrasado PASSED

AntiguedadTest > dias PASSED

AntiguedadTest > horas PASSED

AntiguedadTest > minutos PASSED
```

```
ClienteTest > aceptaUnClienteConTelefono PASSED

ClienteTest > elTelefonoPuedeFaltar PASSED

ClienteTest > noSeConstruyeSinNombre PASSED

ClienteTest > noSeConstruyeConTelefonoVacio PASSED

ProductoTest > noRequiereReposicionEnElStockMinimo PASSED

ProductoTest > requiereReposicionPorDebajoDelStockMinimo PASSED

ProductoTest > noSeConstruyeSinNombre PASSED

ProductoTest > aceptaPrecioCeroParaQueElServidorValideElMinimo PASSED

ProductoTest > noSeConstruyeConStockNegativo PASSED

SesionTest > traduceEstadosDeSesion PASSED

SesionTest > permisoSeConsultaPorAutoridad PASSED

ListarProductosUseCaseTest > devuelveElInventarioDelRepositorio PASSED

ListarProductosUseCaseTest > relanzaLaCancelacionEnLugarDeConvertirlaEnFailure PASSED

ListarProductosUseCaseTest > devuelveFailureCuandoElRepositorioFalla PASSED

RegistrarClienteUseCaseTest > rechazaNombreVacio PASSED

RegistrarClienteUseCaseTest > aceptaUnClienteValido PASSED

RegistrarClienteUseCaseTest > elIdLoAsignaElRepositorio PASSED

RegistrarClienteUseCaseTest > devuelveFailureCuandoElRepositorioFalla PASSED

RegistrarClienteUseCaseTest > rechazaCorreoConFormatoInvalido PASSED

RegistrarClienteUseCaseTest > rechazaTelefonoConMenosDeSeisDigitos PASSED

RegistrarClienteUseCaseTest > elTelefonoEsOpcionalYSeGuardaComoNull PASSED

RegistrarClienteUseCaseTest > reportaLosTresErroresALaVez PASSED

RegistrarProductoUseCaseTest > aceptaStockCero PASSED

RegistrarProductoUseCaseTest > rechazaNombreVacio PASSED

RegistrarProductoUseCaseTest > rechazaNaNComoPrecio PASSED

RegistrarProductoUseCaseTest > rechazaInfinityComoPrecio PASSED

RegistrarProductoUseCaseTest > devuelveFailureCuandoElRepositorioFalla PASSED

RegistrarProductoUseCaseTest > rechazaStockNegativo PASSED

RegistrarProductoUseCaseTest > precioCeroSeDejaLlegarAlServidor PASSED

RegistrarProductoUseCaseTest > rechazaPrecioNoNumerico PASSED

RegistrarProductoUseCaseTest > elIdLoAsignaElRepositorioNoLaPantalla PASSED

RegistrarProductoUseCaseTest > reportaLosTresErroresALaVez PASSED

RegistrarProductoUseCaseTest > aceptaUnPrecioValido PASSED

FormatoTest > formateaSolesConDosDecimales PASSED

ClienteViewModelTest > elTelefonoAusenteSeMuestraComoNoRegistrado PASSED

ClienteViewModelTest > arrancaEnSinClientesCuandoLaCarteraEstaVacia PASSED

ClienteViewModelTest > elCorreoInvalidoCaeEnElFormulario PASSED

ClienteViewModelTest > elClienteRegistradoApareceEnLaCartera PASSED

ProductoViewModelTest > cargaExitosaMuestraLaListaEsperada PASSED

ProductoViewModelTest > consultaLocalMuestraCacheYBloqueaSincronizacionYMutaciones PASSED

ProductoViewModelTest > errorDeRedAlEliminarConservaLaListaPrevia PASSED

ProductoViewModelTest > eliminarSeReflejaPorFlowSinRecargarLaListaYConfirmaExito PASSED

ProductoViewModelTest > productosInactivosNoAparecenEnElInventarioActivo PASSED
```

```
ProductoViewModelTest > listadoVacioTerminaEnSinProductos PASSED

ProductoViewModelTest > compartirEnviaAlCompartidorElTextoArmadoEnCodigoComun PASSED

ProductoViewModelTest > validacionDelServidorSeMuestraEnElFormularioSinCambiarLaFase PASSED

ProductoViewModelTest > precioCeroSeEnviaParaQueLoValideElServidor PASSED

ProductoViewModelTest > cancelarLaCorrutinaNoProduceEstadoFallidoNiMensajeDeError PASSED

SesionViewModelTest > caducarBloqueaAntesDeEsperarHttpYEl401RetiraLaSesionAnterior PASSED

SesionViewModelTest > caducarSinRedMantieneSoloConsultaSinRecuperarPermisos PASSED

[Incubating] Problems report is available at:
file:///H:/ProyectosUPEU/pharmaMobile-/build/reports/problems/problems-report.html

BUILD SUCCESSFUL in 15s
73 actionable tasks: 9 executed, 64 up-to-date
Configuration cache entry reused.
```

Producto 2. Bitácora offline

Cada registro contiene los ocho campos solicitados. Las consultas son copias reales del SQLite del APK debug, leídas con run-as y sqlite3. Las filas inactivas se conservan en la base, mientras la pantalla muestra las activas. Las conclusiones se redactan a partir de lo observado y quedan para revisión personal del estudiante.

E1. Apertura sin red con datos previos

1. Identificador y nombre: E1. Apertura sin red con datos previos

2. Fecha y hora real: 2026-10-23T14:15:20.356694-05:00

3. Plataforma: Android - Pixel 8 Pro, API 37.1

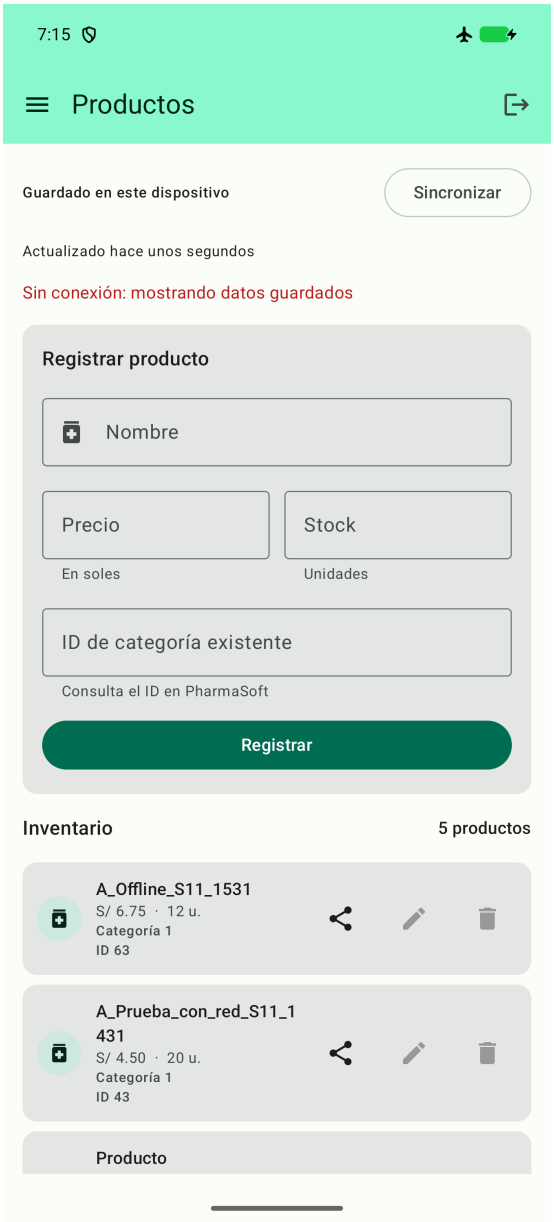
4. Pasos exactos seguidos: Con red: iniciar sesión como admin y abrir Productos; sincronizar. Forzar cierre, activar modo avión y desactivar wifi y datos. Reabrir y entrar a Productos.

5. Qué mostró la aplicación: Inventario visible, cinco productos activos y aviso «Sin conexión: mostrando datos guardados».

6. Qué había en la base local: 9 filas totales, 5 activas, 0 pendientes. Marca de sincronización: [{'id': 1, 'ultimaMillis': 1792782864862}]. Consulta: SELECT id,nombre,precio,stock,pendiente,estado,categoriaId FROM Producto ORDER BY nombre. Archivo: consultas/esc1.json.

7. Captura: capturas/esc1_listado_sin_red.png

8. Conclusión en una línea: Cerrar el proceso no elimina la copia local: el catálogo sigue disponible sin volver a descargarlo.



E2. Registro sin red

1. Identificador y nombre: E2. Registro sin red

2. Fecha y hora real: 2026-10-23T14:16:46.751154-05:00

3. Plataforma: Android - Pixel 8 Pro, API 37.1

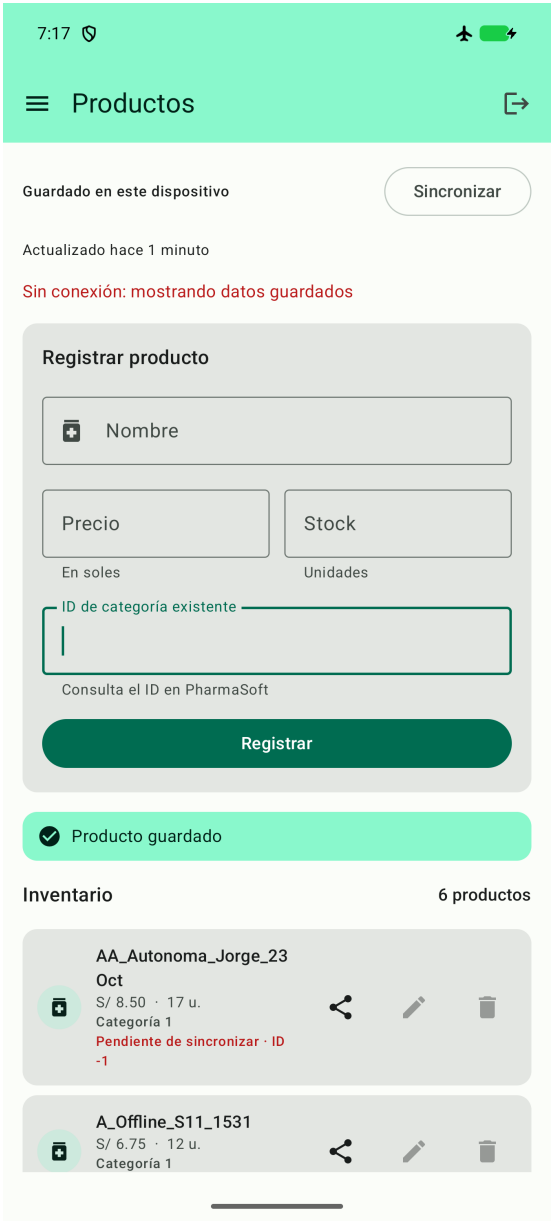
4. Pasos exactos seguidos: Mantener modo avión. Registrar AA_Autonoma_Jorge_23Oct, precio 8.50, stock 17, categoría 1. Cerrar teclado y desplazar el inventario al inicio para ver el nuevo registro.

5. Qué mostró la aplicación: Seis productos activos; el nuevo aparece con «Pendiente de sincronizar · ID -1». Se intentó reconectar antes de tomar la captura final, pero la consulta comprobó que no hubo envío; se volvió a modo avión para documentar el pendiente.

6. Qué había en la base local: 10 filas totales, 6 activas, 1 pendientes. Producto observado: id=-1, pendiente=1, precio=8.5, stock=17. Marca de sincronización: [{'id': 1, 'ultimaMillis': 1792782864862}]. Consulta: SELECT id,nombre,precio,stock,pendiente,estado,categoriaId FROM Producto ORDER BY nombre. Archivo: consultas/esc2_final.json.

7. Captura: capturas/esc2_pendiente_visible.png

8. Conclusión en una línea: El registro se conserva con ID temporal negativo aunque el servidor no pueda recibirlo.



E3. Reconexión y envío

1. Identificador y nombre: E3. Reconexión y envío

2. Fecha y hora real: 2026-10-23T14:17:43.083076-05:00

3. Plataforma: Android - Pixel 8 Pro, API 37.1

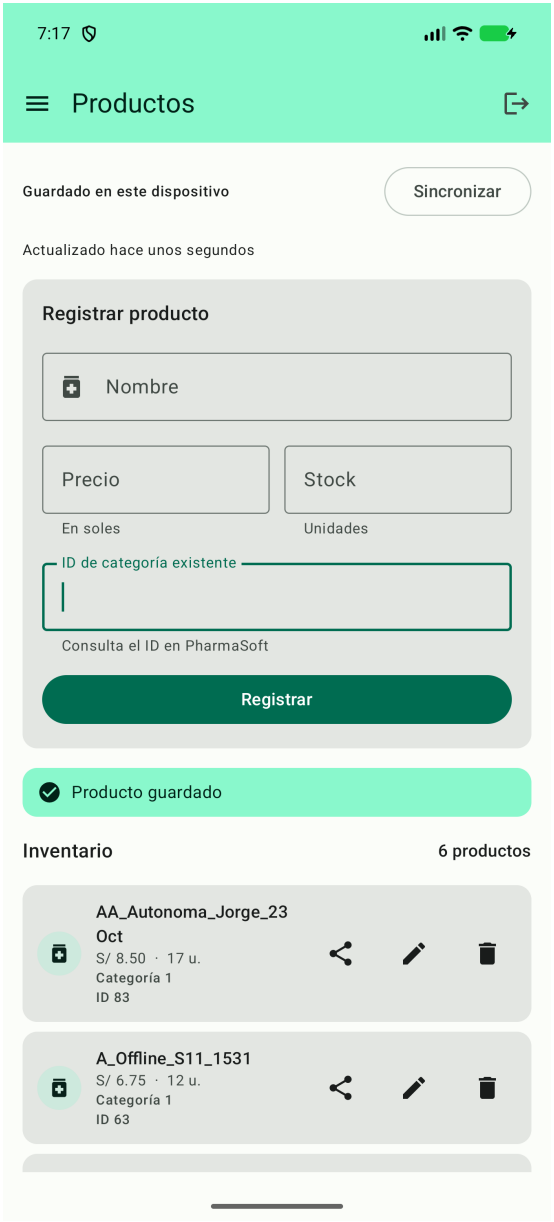
4. Pasos exactos seguidos: Desactivar modo avión, habilitar wifi y datos, esperar conexión y pulsar Sincronizar. En Postman: login de la colección S11 Autonoma Jorge, después GET /api/v1/productos/83.

5. Qué mostró la aplicación: Desaparece la marca pendiente; el producto queda con ID 83. Postman responde 200 OK y devuelve nombre AA_Autonoma_Jorge_23Oct, precio 8.5 y stock 17.

6. Qué había en la base local: 10 filas totales, 6 activas, 0 pendientes. Producto observado: id=83, pendiente=0, precio=8.5, stock=17. Marca de sincronización: [{'id': 1, 'ultimaMillis': 1792783061571}]. Consulta: SELECT id,nombre,precio,stock,pendiente,estado,categoriaId FROM Producto ORDER BY nombre. Archivo: consultas/esc3.json.

7. Captura: capturas/esc3_producto_sincronizado.png

8. Conclusión en una línea: El GET independiente de Postman confirma que el producto llegó a PharmaSoft y recibió el ID 83.



E4. Primera apertura sin red

1. Identificador y nombre: E4. Primera apertura sin red

2. Fecha y hora real: 2026-10-23T14:19:04.658576-05:00

3. Plataforma: Android - Pixel 8 Pro, API 37.1

4. Pasos exactos seguidos: Confirmar cero pendientes y respaldar SQLite. Forzar cierre, activar modo avión, ejecutar pm clear pe.edu.upeu.pharmamobil y abrir la app.

5. Qué mostró la aplicación: «Todavía no hay productos», explicación para conectarse e iniciar sesión y «Aún no se ha sincronizado». No se presenta una pantalla de error.

6. Qué había en la base local: 0 filas totales, 0 activas, 0 pendientes. Marca de sincronización: []. Consulta: SELECT id,nombre,precio,stock,pendiente,estado,categoriaId FROM Producto ORDER BY nombre. Archivo: consultas/esc4.json.

7. Captura: capturas/esc4_primera_apertura_sin_red.png

8. Conclusión en una línea: Una instalación sin caché informa que no tiene datos en vez de aparentar un fallo del inventario.



E5. Sesión expirada sin red

1. Identificador y nombre: E5. Sesión expirada sin red

2. Fecha y hora real: 2026-10-23T14:22:58.982742-05:00

3. Plataforma: Android - Pixel 8 Pro, API 37.1

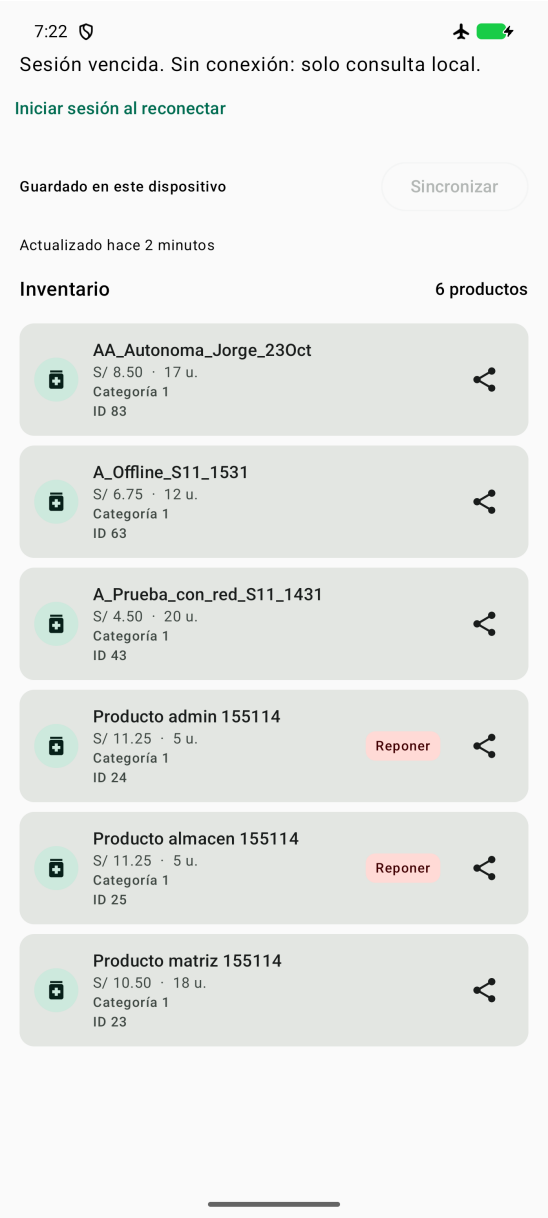
4. Pasos exactos seguidos: Arrancar temporalmente el backend con APP_JWT_EXPIRACION_MINUTOS=1. Iniciar sesión en Android, sincronizar y cerrar. Esperar el vencimiento emitido por el backend. Activar modo avión, forzar cierre y comprobar metadatos antes de reabrir. Abrir ya con jwt_exp=1792783267 y ahora=1792783361. Después reconectar y comprobar el login.

5. Qué mostró la aplicación: «Sesión vencida. Sin conexión: solo consulta local», seis productos activos, sin formulario ni botones de edición/eliminación y Sincronizar deshabilitado. El token se borra. Al reconectar se solicita iniciar sesión nuevamente.

6. Qué había en la base local: 10 filas totales, 6 activas, 0 pendientes. Producto observado: id=83, pendiente=0, precio=8.5, stock=17. Marca de sincronización: [{'id': 1, 'ultimaMillis': 1792783225926}]. Consulta: SELECT id,nombre,precio,stock,pendiente,estado,categoriaId FROM Producto ORDER BY nombre. Archivo: consultas/esc5_final.json.

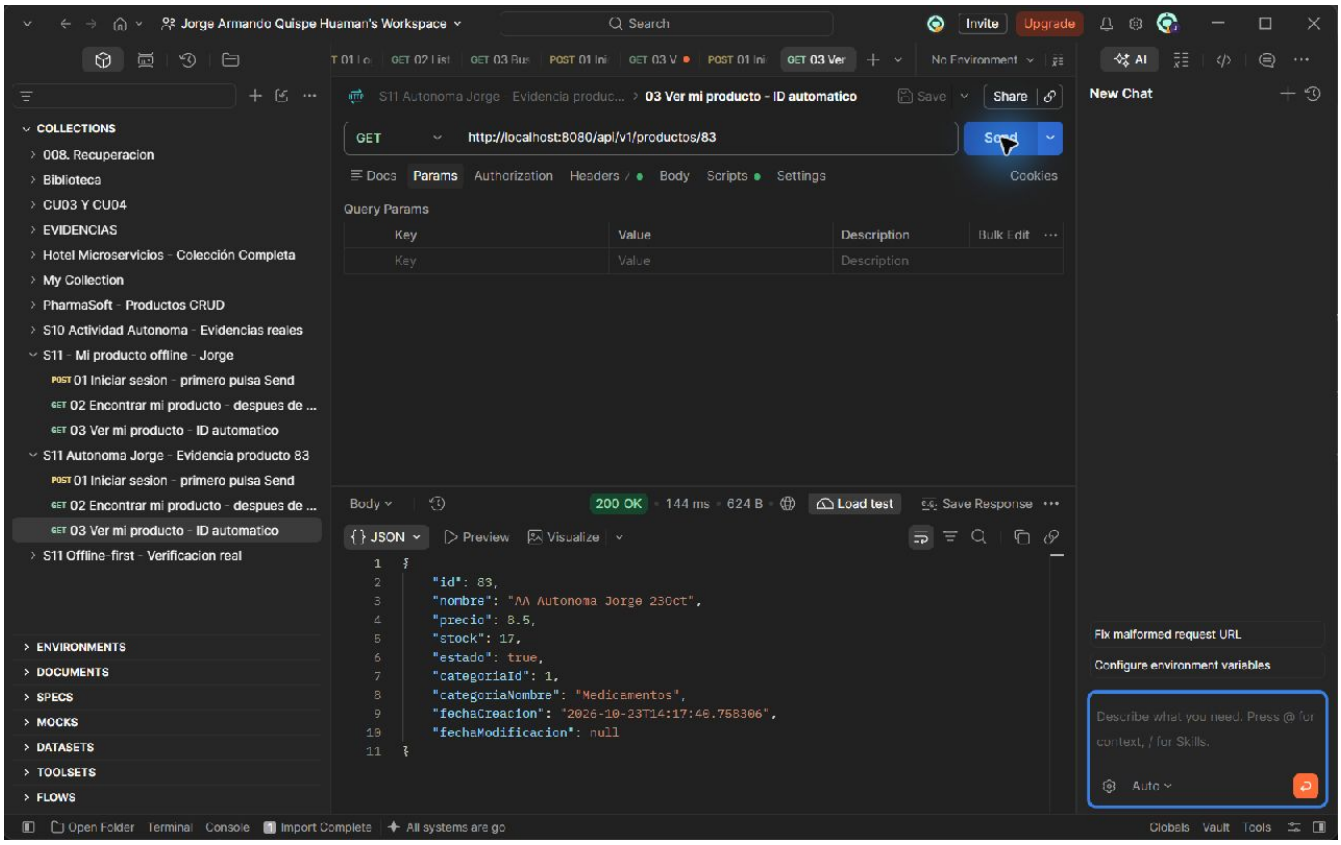
7. Captura: capturas/esc5_consulta_local_verificada.png

8. Conclusión en una línea: Vencer la autorización bloquea los cambios, pero no impide consultar el inventario ya guardado en este dispositivo.



E3. Confirmación desde Postman

GET http://localhost:8080/api/v1/productos/83. Respuesta real 200 OK. Este cliente consulta el backend; ver el producto solo en Android no bastaría para comprobar el envío. Postman se emplea por solicitud expresa del usuario, en sustitución de Swagger.



Datos devueltos por el servidor

```
{
  "id": 83,
  "nombre": "AA_Autonoma_Jorge_23Oct",
  "precio": 8.5,
  "stock": 17,
  "estado": true,
  "categoriaId": 1,
  "categoriaNombre": "Medicamentos",
  "fechaCreacion": "2026-10-23T14:17:40.758306",
  "fechaModificacion": null
}
```

La colección entregada no incluye token ni contraseña: se configura claveAdmin durante la sesión y se ejecuta el login para obtener JWT. La captura muestra el producto, no las credenciales.

E5. Argumento y reconexión

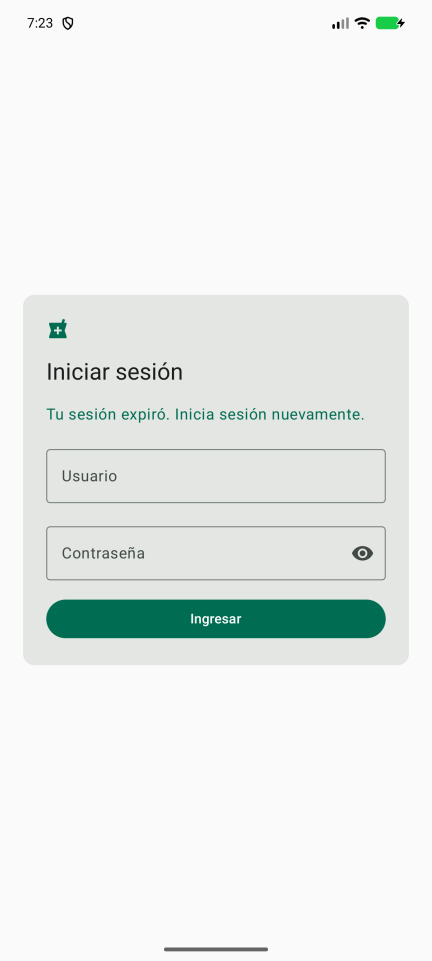
El login sin internet impedía consultar datos ya guardados. La versión anterior se documentó con captura, sin atribuir a ese token una expiración no medida. En la prueba controlada final se emitió un JWT real con duración de un minuto y se comprobó su vencimiento antes de reabrir; no se alteró la firma.

```
10-23 19:20:27.494  9649  9649 I AuthDebugReceiver: METADATA token_presente=true jwt_exp=1792783267 ahora=1792783227
vencido=false sesion_exp=1792783267

10-23 19:22:41.318 11058 11058 I AuthDebugReceiver: METADATA token_presente=true jwt_exp=1792783267 ahora=1792783361
vencido=true sesion_exp=1792783267

10-23 19:22:57.989 11058 11058 I AuthDebugReceiver: METADATA token_presente=false jwt_exp=null ahora=1792783377
vencido=false sesion_exp=null
```

El primer intento abrió antes del vencimiento y no se usa como evidencia final. Se repitió después de verificar exp < ahora. La consulta conserva diez filas, seis activas, sin conceder sesión ni escritura. Al reconectar la app exige iniciar sesión. Consultar una copia local con su antigüedad visible es más útil que exigir un login imposible sin red; no equivale a renovar la autorización.



Producto 3. Antigüedad

Sincronizacion contiene una sola fila, id=1 y ultimaMillis. ProductoRepositorioOffline guarda la fecha únicamente después de subir todos los pendientes y descargar todas las páginas. La lista y la marca se confirman en una transacción. Un fallo conserva la fecha anterior. El reloj se puede inyectar para probar sin esperas.

La migración 1.sqm pasó la instalación existente de esquema 1 a 2 sin alterar ninguno de sus nueve productos. Ver consultas/migracion-verificada.json. No se borró la base para comprobar la migración.

Función pura de presentación

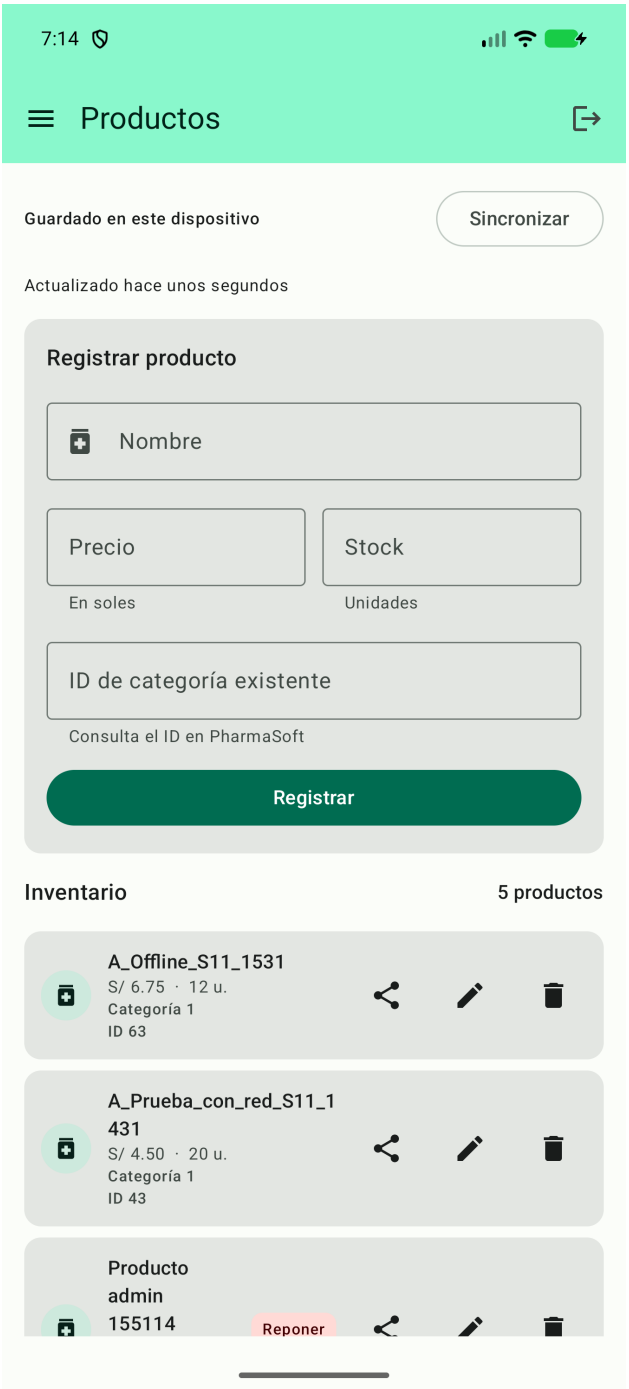
```
package pe.edu.upeu.pharmamobil.domain.model

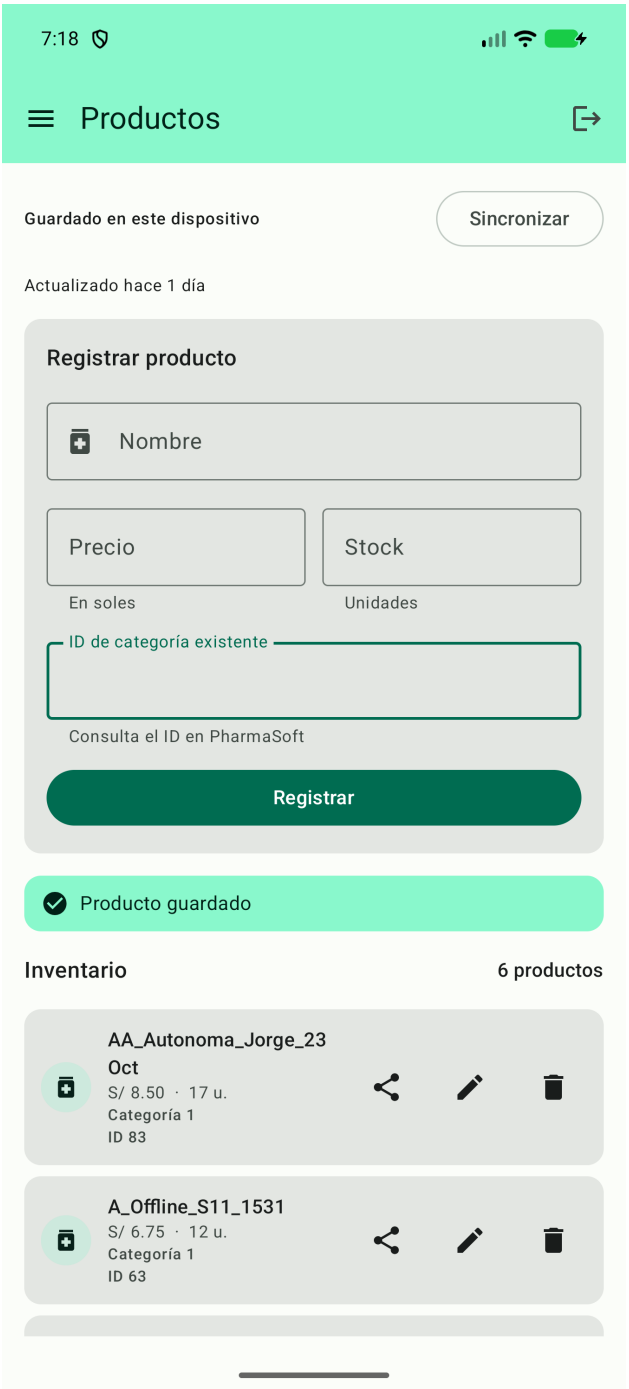
/** Recibe el reloj como dato: permite comprobar los límites sin esperar tiempo real. */
fun formatearAntiguedad(ahora: Long, ultima: Long?): String {
    if (ultima == null) return "Aún no se ha sincronizado"
    val minutos = ((ahora - ultima).coerceAtLeast(0) / 60_000)
    return when {
        minutos == 0L -> "Actualizado hace unos segundos"
        minutos < 60 -> "Actualizado hace $minutos ${if (minutos == 1L) "minuto" else "minutos"}"
        minutos < 1440 -> (minutos / 60).let { "Actualizado hace $it ${if (it == 1L) "hora" else "horas"}" }
        else -> (minutos / 1440).let { "Actualizado hace $it ${if (it == 1L) "día" else "días"}" }
    }
}
```

Se probaron nulo, segundos, reloj atrasado, minutos, horas y días. La marca se observa por Flow y la interfaz recalcula el texto cada minuto, sin peticiones HTTP adicionales. Sin marca aparece «Aún no se ha sincronizado»; véase E4.

Antigüedad reciente y controlada

Izquierda: sincronización real reciente. Derecha: marca modificada SOLO en el APK debug a hace 26 horas mediante DEBUG_ANTIGUEDAD_26_HORAS. No se esperó un día real, no se insertaron filas de productos y no se cambió el reloj del emulador. Después se restauró la marca mediante una sincronización exitosa; consta en consultas/antes_borrar_datos.json y final_restaurado.json.





Producto 4. Guion de ocho minutos

Guion de sustentación - ocho minutos

Preparación previa

Backend oficial de laboratorio en localhost:8080, Oracle en pharmasoft-oracle (puerto 1522), AVD Pixel_8_Pro abierto, modo avión desactivado, sesión Android cerrada antes de empezar y Postman abierto. Comprobar GET /api/health y categoría 1. Usuarios reales: admin con escritura y vendedor sin CATALOGO_ESCRIBIR; para el fallo usar admin con contraseña incorrecta. Ingresar claves únicamente durante la sesión; este documento no las contiene. Android usa 10.0.2.2:8080; Postman usa localhost:8080. El emulador no es necesario para consultar el servidor con Postman.

Postman sustituye Swagger por decisión expresa del usuario. iOS se declara sin evidencia. Preparar peticiones de login admin/vendedor, GET del producto y PUT restringido sin ejecutar altas o bajas ajenas. Usar un nombre nuevo en cada ensayo. Antes de empezar cerrar sesión desde la app para mostrar autenticación.

0-1 min. Autenticación JWT

Qué digo: «La app obtiene un JWT al iniciar sesión y lo guarda cifrado. Una contraseña incorrecta no concede acceso.»

Qué hago: Desde el login, demostrar primero el rechazo; después entrar correctamente. Alternativa de orden para no perder tiempo: login correcto, cerrar sesión, fallo y volver a entrar.

Usuario: admin; repetir admin con clave deliberadamente incorrecta.

Riesgo y plan B: El backend apagado produce error de conexión, no un rechazo de credenciales. Plan B: comprobar salud, recuperar servicio y repetir; no presentar el error de red como 401.

1-3 min. Consumo REST y CRUD

Qué digo: «La interfaz consulta SQLite; la sincronización envía primero los pendientes y después descarga todas las páginas del servidor.»

Qué hago: Abrir Productos, listar, crear un producto de demostración con nombre único, precio 5.50, stock 15, categoría 1. Mostrar ID positivo, editar precio y stock y verificar el cambio. Si el docente pide eliminar, dar de baja solo el producto de demostración; se trata de baja lógica.

Usuario: admin.

Riesgo y plan B: Categoría inexistente o nombre repetido impiden el alta. Plan B: categoría 1 comprobada y nombre distinto; no modificar productos del negocio.

3-5 min. Estados y errores

Qué digo: «Ocultar una acción en la app no basta: el backend también comprueba permisos. Los datos inválidos se señalan debajo de su campo.»

Qué hago: En Postman obtener JWT de vendedor y solicitar PUT al producto 83: debe responder 403 sin modificarlo. Volver a Android como admin, pulsar Registrar con campos vacíos y mostrar sus mensajes. La app oculta edición al vendedor; por eso el rechazo explícito se demuestra en Postman.

Usuario: vendedor para 403; admin para validación.

Riesgo y plan B: Sin token se obtiene 401, no 403. Plan B: rehacer login de vendedor y comprobar Authorization. No cambiar la aserción para aceptar otro estado.

5-7 min. Offline-first

Qué digo: «Estos datos vienen de la base del teléfono. El producto local recibe un ID negativo y se confirma con el ID del servidor cuando vuelve la conexión.»

Qué hago: Sincronizar, activar modo avión y abrir catálogo; registrar otro producto único y mostrar Pendiente. Desactivar modo avión, esperar conexión, pulsar Sincronizar y comprobar el ID en Postman. Usar producto 83 como evidencia ya documentada si hace falta.

Usuario: admin con sesión vigente.

Riesgo y plan B: El teclado puede ocultar la lista: cerrarlo y desplazar el inventario al inicio. Si vence la sesión, volver a iniciar sesión antes de enviar; la consulta local no permite escrituras.

7-8 min. Evidencia multiplataforma

Qué digo: «La lógica del repositorio y sus pruebas está en commonMain y commonTest. Aquí se ejecutó Android. No se adjunta una ejecución de iOS.»

Qué hago: Mostrar pruebas comunes, reporte de 94 casos y pantalla Android. El requisito original pide Android e iOS; este tramo es parcial porque se excluyó iOS y el equipo es Windows sin Xcode. No afirmar que pasó en iOS.

Usuario: admin en Android.

Riesgo y plan B: No hay plan B que equivalga a ejecutar iOS en este equipo. Para acreditar ese criterio completo se necesita un Mac y repetir la demostración. El estudiante decidió mantener el alcance Android.

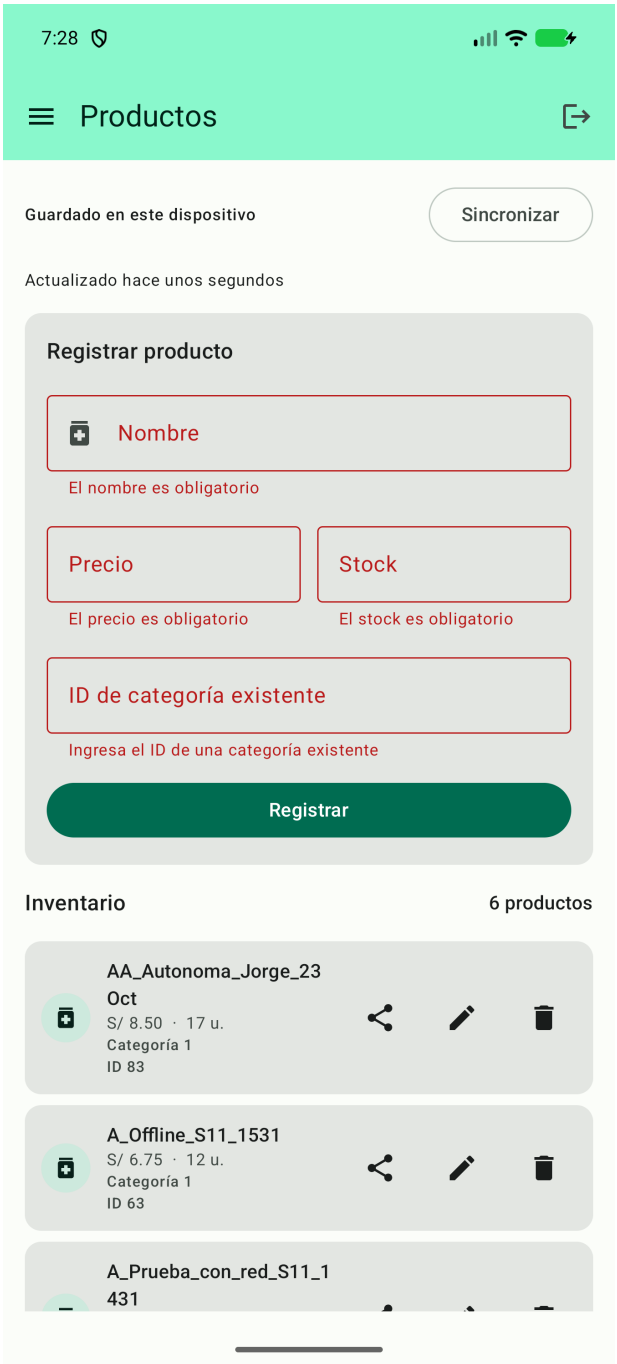
Comprobaciones y tareas personales

En comprobacion-defensa.json constan login correcto 200, incorrecto 401, PUT vendedor 403 y validación 400. La captura validacion_bajo_campos.png muestra errores reales en Android. Las acciones REST/Flow cuentan con pruebas del proyecto. El recorrido completo no se ha ensayado en voz alta ni se garantiza que la persona lo complete en ocho minutos: debe ensayarlo Jorge con cronómetro, estudiar cada línea, revisar conclusiones y preparar la demostración del 27 de octubre.

Comprobaciones para la sustentación

Ejecución real del 23/10/2026: login admin 200; login vendedor 200; clave incorrecta 401; PUT de vendedor al producto 83, 403; POST inválido de admin, 400. No se aceptó un 401 como prueba de permisos ni se modificó el producto al recibir 403. El backend se restauró a JWT de 3600 segundos. Archivo: docs/s11/comprobacion-defensa.json.

En Android se pulsó Registrar con el formulario vacío. Los mensajes se muestran bajo sus campos y no se insertó ningún registro inválido. Esta captura complementa el tramo de manejo de estados; no sustituye el ensayo personal con cronómetro.



Lista de cotejo y alcance real

1. PDF con nombre S11_ActividadAutonoma_Quispe_Huaman.pdf: Sí.
2. Portada: nombre completo, sesión, fecha y enlace a rama: Sí; enlace publicado y verificado.
3. Seis pruebas en commonTest y todas pasan: Sí.
4. Salida completa de la ejecución adjunta: Sí.
5. Cinco escenarios con capturas respectivas: Sí.
6. Escenario 3 con evidencia desde Swagger: No: Postman por instrucción del usuario.
7. Escenario 5 argumenta el comportamiento observado: Sí; revisión personal pendiente.
8. Pantalla indica última sincronización correcta: Sí.
9. Caso nunca sincronizado resuelto: Sí.
10. Guion cubre los cinco criterios de Producto U2: No completamente: iOS excluido; Android documentado.
11. Commits accesibles desde el enlace entregado: Sí; rama publicada y enlace verificado.

Revisión de la rúbrica

Suite (7 pts): nivel logrado por las seis pruebas y salida completa. Bitácora (6 pts): cinco escenarios con datos reales; la sustitución de Swagger por Postman puede afectar el requisito literal. Antigüedad (3 pts): implementada y comprobada. Guion (2 pts): cinco tramos preparados; evidencia iOS pendiente. Análisis y repositorio (2 pts): conclusiones específicas y commits publicados; acceso remoto verificado. La nota definitiva corresponde al docente; no se afirma 20/20 ni 100 % de cumplimiento.

Pendiente de Jorge: revisar conclusiones, estudiar el código, ensayar con cronómetro, subir PDF y enlace al aula antes del lunes 26/10/2026 a las 23:59 y sustentar el martes 27/10. El ensayo y la entrega no se simulan.

Explicación para estudiar

Guía de estudio: cómo explicar lo agregado

Producto 1: seis pruebas

Archivo: shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobil/data/repository/ProductoRepositorioOfflineAutonomaTest.kt.

Preparación, línea por línea útil

- nuevo: construye el producto de entrada con ID 0. El repositorio asignará otro ID; la pantalla no lo decide.
- cabeceras: indica que las respuestas simuladas contienen JSON.
- Producto.dto(): transforma el producto de dominio al contrato real que devuelve PharmaSoft.
- pagina(...): serializa una página completa con la lista, cantidad y bandera ultima=true. No es una petición real.
- conBaseNueva(...): recibe un motor HTTP y el bloque que comprobará un caso.
- crearDriverEnMemoria(): crea SQLite temporal mediante expect/actual; JDBC solo aparece en androidHostTest.
- baseEnMemoria(driver): crea el esquema y entrega PharmaDb; cada llamada tiene una base independiente.
- ProductoLocal(...): permite consultar y observar esa base.
- crearHttpClient(... engine=motor): sustituye el transporte con MockEngine sin cambiar el código del repositorio.
- try { comprobar(...) }: ejecuta las aserciones con repositorio y base del mismo caso.
- finally: cierra cliente y driver incluso cuando una aserción falla.
- @Test: hace que Gradle descubra el caso. runTest controla corrutinas con un planificador de pruebas.
- first(): recoge la primera lista emitida por el Flow; no espera con sleep.
- assertEquals: compara lo esperado y lo obtenido. assertTrue: exige una condición, no la imprime solamente.

Prueba 2: registro sin red

- MockEngine { throw IOException(...) }: simula que el transporte no puede alcanzar el servidor.
- repo.registrar(nuevo): debe guardar primero en SQLite aunque el envío falle.
- repo.observar().first().single(): obtiene el único producto del listado local.
- guardado.id < 0: demuestra el identificador temporal.
- emitido.pendiente: exige que no se confunda con un envío confirmado.
- assertEquals(guardado, emitido): compara todos sus campos.
- local.pendientes(): comprueba directamente la fila que deberá enviarse al reconectar.

Prueba 4: la bajada conserva pendientes

- Se registra sin red y se prepara otro producto con ID 71.
- local.reemplazarSincronizados(listOf(remoto)): aísla la operación de descarga. Un POST fallido detiene la sincronización completa antes de descargar; por eso este caso comprueba directamente el reemplazo local.
- Cantidad 2: deben permanecer el pendiente y el remoto.
- Conjunto de IDs: exige ambos identificadores sin depender del orden alfabético.
- Comparar el producto pendiente completo: detecta si se perdió su marca o cambió algún dato.
- local.pendientes(): debe seguir conteniendo exactamente la fila original.

Prueba 1: sincronización con red

- Se preparan dos productos del servidor con IDs 41 y 42.
- El motor exige método GET y devuelve una página JSON completa.
- repo.sincronizar(): descarga y confirma la lista en SQLite.
- Comparación de conjuntos: los datos emitidos deben coincidir con todos los productos del servidor.
- Cantidad 2, igualdad con local.listar() y pendientes vacíos: contrastan Flow, persistencia y estado de envío.

Prueba 3: envío al reconectar

- conRed=false: la misma base inicia desconectada.
- postAceptados=0: cuenta únicamente POST aceptados por el motor.
- El motor rechaza sin red; con red devuelve el producto confirmado con ID 72 para POST y la página para GET.
- Se registra un pendiente y se comprueba su ID negativo.
- conRed=true y repo.sincronizar(): cambian el transporte sin reemplazar la base.
- POST igual a 1, ID 72 y marca pendiente falsa: acreditan el envío y su confirmación.
- Pendientes vacíos: ya no debe enviarse nuevamente ese registro.

Prueba 5: identificadores sin colisión

- El producto existente del servidor usa ID 41; el local recibe ID negativo.
- Tras el POST, el nuevo obtiene ID 72; la descarga contiene los dos.
- Se exigen dos filas y ambos nombres y valores: contar filas sin comprobar contenido podría ocultar una sobrescritura.

Prueba 6: segunda sincronización

- Se registra sin red y luego se habilita el motor.
- Se sincroniza dos veces sobre la misma base.
- Se comparan las dos listas completas y sus cantidades.
- Se exige un único POST en total y ningún pendiente. Una segunda subida o duplicación hace fallar el caso.

Producto 3: antigüedad, línea por línea

- Sincronizacion.sq: declara tabla adicional; id=1 restringe a una fila. ultimaMillis almacena epoch en milisegundos, no un texto ya formateado.
- leerUltima: lee solo esa marca; si no existe devuelve null mediante executeAsOneOrNull.
- guardarUltima: inserta o reemplaza la única fila.
- 1.sqm: crea la tabla cuando una instalación pasa de esquema 1 a 2; no borra productos.
- observarUltimaSincronizacion: convierte la consulta SQL en Flow. La interfaz recibe cambios automáticamente.
- confirmarSincronizacion: engloba reemplazo y fecha en una transacción para no actualizar solo una mitad.
- ahora: () -> Long: inyecta un reloj. En producción consulta Clock.System; el test entrega un número fijo.
- El repositorio llama esa transacción después de enviar pendientes y terminar todas las páginas. Si algo falla antes, la marca no cambia.
- formatearAntiguedad(ahora, ultima): función pura; la misma entrada siempre da el mismo texto.
- ultima == null: devuelve «Aún no se ha sincronizado».
- coerceAtLeast(0): un reloj atrasado no muestra antigüedad negativa.
- Dividir entre 60000 convierte milisegundos en minutos enteros.
- Cero minutos: segundos; menos de 60: minutos; menos de 1440: horas; el resto: días.
- Las condiciones de singular evitan «1 minutos» o «1 días».
- ProductoViewModel recoge el Flow y copia la fecha al estado de pantalla.
- ProductoScreen recuerda el instante actual. Su LaunchedEffect actualiza el texto cada minuto; ese delay pertenece a la interfaz de producción, no a las seis pruebas ni a HTTP.
- Las cinco pruebas del formateador cubren nulo, segundos, minutos, horas, días y reloj atrasado. La prueba del repositorio exige conservar la marca tras un fallo.

Producto 2: sesión vencida y evidencia

- ConsultaLocalException: señala que solo puede consultarse SQLite. No es un objeto Sesion ni concede permisos.
- sesionGuardada: intenta comprobar auth/me también cuando no hay token. Con red, el 401 conduce al login; sin red, el fallo de transporte permite consulta local.
- guardada: solo acepta la sesión cifrada que corresponde al token almacenado.
- sesionSinRed: si la sesión sigue vigente conserva el comportamiento previo. Si venció o no existe, borra credenciales y solicita consulta local.
- consultaLocal y sesionLocalExpirada: son estado de interfaz, no permisos del servidor.
- SesionViewModel recibe la excepción y deja sesion=null; mantiene la bandera de consulta.
- El ViewModel de productos de consulta usa otra clave de Koin. No inicia sincronización.
- iniciarOperacion y sincronizar: retornan inmediatamente en modo consulta; no basta con esconder un botón.
- ProductoScreen sin escritura omite el formulario y los controles de editar/eliminar; el botón de sincronizar queda deshabilitado.
- App comprueba conexión cada cinco segundos mientras está en consulta. Cuando auth/me vuelve a responder pide login. La comprobación de la fecha también se programa si la sesión vence con la app abierta.
- DEBUG_METADATA_SESION: solo en APK debug; lee exp del JWT y registra fechas, nunca el token.
- DEBUG_ANTIGUEDAD_26_HORAS: modifica únicamente la marca de sincronización en depuración; no inserta productos ni cambia el reloj. La captura declara el método.
- consultar_base.py: copia el archivo real del emulador con run-as, lo abre con SQLite y guarda el resultado de SELECT en JSON. No agrega filas.
- Las capturas son screencap de Android y captura nativa de Postman. Los JSON describen lo consultado en el instante de cada escenario.

Producto 4 y revisión personal

El guion indica lo que diré y haré, usuario, tiempo y plan B por tramo. Debo practicarlo realmente con cronómetro y entender por qué un 401 es distinto de un 403. Postman es otro cliente del backend: no necesita que el emulador esté corriendo. Docker mantiene Oracle y el backend atiende HTTP; SQLDelight conserva los datos del teléfono cuando esos servicios no se pueden alcanzar.

Debo revisar las conclusiones de los escenarios antes de asumirlas como propias. No se ejecutó iOS ni se simuló el ensayo. Los cambios están publicados en mi rama de GitHub, tras mi confirmación final. La entrega al aula y la sustentación las realizo personalmente.

- Refuerzo final: SesionViewModel retira inmediatamente una sesión ya vencida antes de esperar HTTP. Un rechazo 401 no conserva el objeto Sesion anterior. AuthRepositorioRest borra el token vencido antes de auth/me; el nuevo test exige ausencia de Authorization. Las dos pruebas del ViewModel controlan la respuesta con CompletableDeferred, sin dormir ni usar red.